
Calclatorz

Arithmetic Expression Evaluator in C++

Test Cases For Evaluator Module

Version 1.1

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

Revision History

Date	Version	Description	Author
2026-05-01	1.0	Initial draft	Eian
2026-05-06	1.1	Final polishing	Isaiah

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

Table of Contents

1. Purpose	4
2. Basic Addition (Stack Mechanics)	4
3. Basic Subtraction (Operand Order)	5
4. Chained Operations (Multi-Operator Expression)	6
5. Floating Point Operands	7
6. Division By Zero	8
7. Too Few Operands	9
8. Missing Operator	10
9. Unknown Token in Switch Default	11
10. Single Number (Minimum Valid Expression)	12
11. Negative Result from Subtraction	13

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

Test Cases

1. Purpose

This *Test Cases* specification document for the Arithmetic Expression Evaluator in C++ defines the test cases for the `evaluator` module. Specifically, it covers the `evaluatePostfix()` function, which processes a vector of postfix-notation tokens and returns a computed double result. The cases address black-box behavior, white-box code paths, and boundary values.

2. Basic Addition (Stack Mechanics)

2.1 Test case identifier

ETC-001

2.2 Test item

Plus operator branch. Verifies that two integer operators are correctly summed and the result is returned.

2.3 Input specifications

A postfix token list: [Token(Number, "3"), Token(Number, "4"), Token(Plus)]

2.4 Output specifications

Return value: 7.0 (double). No exception thrown.

2.5 Environmental needs

C++ Compiler

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

3. Basic Subtraction (Operand Order)

3.1 Test case identifier

ETC-002

3.2 Test item

Minus operator branch. Verifies that left and right operators are popped in the correct order so that left to right is computed, not right to left.

3.3 Input specifications

A postfix token list: [Token(Number, "10"), Token(Number, "3"), Token(Minus)]

3.4 Output specifications

Return value: 7.0 (double). No exception thrown.

3.5 Environmental needs

C++ Compiler

3.6 Special procedural requirements

Special attention to the stack pop order, right is popped first, then left.

3.7 Intercase dependencies

ETC-001 should pass first to confirm basic stack mechanics are working

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

4. Chained Operations (Multi-Operator Expression)

4.1 Test case identifier

ETC-003

4.2 Test item

Multiple operators in sequence. Verifies that the stack is managed correctly across more than one operator, producing the right intermediate and result.

4.3 Input specifications

A postfix token list representing $2 + (3 * 4)$: [Token(Number, "2"), Token(Number, "3"), Token(Number, "4"), Token(Multiply), Token(Plus)]

4.4 Output specifications

Return value: 14.0 (double). No exception thrown.

4.5 Environmental needs

C++ Compiler

4.6 Special procedural requirements

Intermediate value 12.0 ($3 * 4$) must remain on stack and be consumed by the Plus operator.

4.7 Intercase dependencies

ETC-001, ETC-002.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

5. Floating Point Operands

5.1 Test case identifier

ETC-004

5.2 Test item

std::stod parsing and Multiply branch. Verifies that decimal string tokens are parsed correctly and that multiplication produces the right floating-point result.

5.3 Input specifications

A postfix token list: [Token(Number, "1.5"), Token(Number, "2.5"), Token(Multiply)]

5.4 Output specifications

Return value: 3.75 (double). No exception thrown.

5.5 Environmental needs

C++ Compiler

5.6 Special procedural requirements

Result comparison should use a tolerance epsilon (e.g. 1e-9) rather than exact equality due to floating-point representation.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

6. Division By Zero

6.1 Test case identifier

ETC-005

6.2 Test item

Divide branch, zero denominator guard. Verifies that dividing by exactly 0.0 throws the expected runtime error rather than producing undefined behavior.

6.3 Input specifications

A postfix token list: [Token(Number, "5"), Token(Number, "0"), Token(Divide)]

6.4 Output specifications

std::runtime_error thrown with message: "Division by zero". No return value.

6.5 Environmental needs

C++ Compiler

6.6 Special procedural requirements

Test must assert both that an exception is thrown AND that its message matches "Division by zero".

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

7. Too Few Operands

7.1 Test case identifier

ETC-006

7.2 Test item

Values.size() < 2 guard. Verifies that providing an operator without enough preceding operands throws the expected error instead of causing undefined stack behavior.

7.3 Input specifications

A postfix token list: [Token(Number, "5"), Token(Plus)]

7.4 Output specifications

std::runtime_error thrown with message: "Invalid expression". No return value.

7.5 Environmental needs

C++ Compiler

7.6 Special procedural requirements

White-box: directly targets the size < 2 check before the operator switch.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

8. Missing Operator

8.1 Test case identifier

ETC-007

8.2 Test item

values.size() != 1 final check. Verifies that a token list with two numbers and no operator leaves two values on the stack and triggers the final validation error.

8.3 Input specifications

A postfix token list: [Token(Number, "3"), Token(Number, "4")]

8.4 Output specifications

std::runtime_error thrown with message: "Invalid expression". No return value.

8.5 Environmental needs

C++ Compiler

8.6 Special procedural requirements

White-box: directly targets the post-loop stack-size assertion.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

9. Unknown Token in Switch Default

9.1 Test case identifier

ETC-008

9.2 Test item

Default branch of the operator switch. Verifies that an unrecognized token type (e.g. a left-parenthesis or identifier token that somehow reaches the evaluator) causes a descriptive error rather than silent incorrect evaluation.

9.3 Input specifications

A postfix token list: [Token(Number, "3"), Token(Number, "4"), Token(LeftParen)] — where LeftParen is a TokenType not handled by any case.

9.4 Output specifications

std::runtime_error thrown with message: "Unexpected token in postfix evaluation". No return value.

9.5 Environmental needs

C++ Compiler

9.6 Special procedural requirements

Requires constructing a Token with an unsupported TokenType value. Verify this is possible without triggering undefined behavior.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

10. Single Number (Minimum Valid Expression)

10.1 Test case identifier

ETC-009

10.2 Test item

Boundary case where the token list contains exactly one Number token and no operators. Verifies the post-loop check accepts a stack of size 1 and returns the value directly.

10.3 Input specifications

A postfix token list: [Token(Number, "42")]

10.4 Output specifications

Return value: 42.0 (double). No exception thrown.

10.5 Environmental needs

Specify any other requirement.

10.6 Special procedural requirements

This is the minimum non-empty valid input. Confirms the evaluator does not require at least one operator to succeed.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Evaluator	Date: 2026-05-06

11. Negative Result from Subtraction

11.1 Test case identifier

ETC-010

11.2 Test item

Minus operator branch, boundary value producing a negative result. Verifies that operand ordering is correct (left=3, right=10) and that the evaluator handles a negative double return value without error.

11.3 Input specifications

A postfix token list: [Token(Number, "3"), Token(Number, "10"), Token(Minus)]

11.4 Output specifications

Return value: -7.0 (double). No exception thrown.

11.5 Environmental needs

C++ Compiler

11.6 Special procedural requirements

If this test returns 7.0 instead of -7.0, it indicates the pop order is reversed (right - left instead of left - right).

11.7 Intercase dependencies

ETC-002 should pass first to confirm basic subtraction ordering.